BUG REPORT - Ruts-Walking-Street Project

Date: 23/08/2026 19:48:02

Executive Summary

Total Bugs Found	9 issues
CRITICAL Severity	1
HIGH Severity	3
MEDIUM Severity	4
LOW Severity	1

BUG #1: Multiple Cross-Site Scripting (XSS) Vulnerabilities

Severity: CRITICAL | **File:** script.js

Problem: Application uses innerHTML to inject unsanitized data from users and database, allowing attackers to inject malicious HTML/JavaScript.

• Line 45: toast.innerHTML from msg parameter directly
• Line 63: showConfirm() injects msg without sanitization
• Line 938: shopName embedded in onclick attribute
• Lines 1283, 1342: Database values injected via innerHTML

Impact: Attackers can steal cookies, sessions, or perform phishing attacks

Recommended Fix: Replace all innerHTML with textContent or use proper HTML escaping

BUG #2: Unvalidated URL Injection in onclick Handler

Severity: HIGH | **File:** script.js, Line 1416

Problem: Image URLs directly injected into onclick attributes without quote escaping. URLs containing quotes can break out and inject code.

Recommended Fix: Use data attributes with addEventListener instead of inline event handlers

BUG #3: Missing HTTP Response Status Check

Severity: HIGH | **File:** script.js, Line 308

Problem: Code calls await uploadRes.json() without checking if HTTP response was successful (status 200-299).

Impact: Server errors (HTTP 500) may cause JSON parsing failure with unclear error messages

Recommended Fix: Check uploadRes.ok or uploadRes.status before parsing JSON

BUG #4: Incorrect Content-Type Header

Severity: MEDIUM | **File:** script.js, Lines 306 & 1268

Problem: JSON data is sent with Content-Type: text/plain header instead of application/json

Recommended Fix: Add proper header: { "Content-Type": "application/json" }

BUG #5: Race Condition in Image Upload

Severity: MEDIUM | **File:** script.js, Lines 233-240

Problem: Multiple FileReader operations are initiated without waiting for completion. Form can be submitted before all images finish loading.

Recommended Fix: Use Promise.all() to ensure all images are loaded before proceeding

BUG #6: Missing Status Check in deleteGoogleDriveFolder

Severity: MEDIUM | **File:** script.js, Lines 1267-1273

Problem: Fetch response JSON is parsed without checking HTTP response status first

Recommended Fix: Check res.ok before calling res.json()

BUG #7: DOM Clobbering via onclick Attribute

Severity: HIGH | **File:** script.js, Line 939

Problem: Shop names are embedded directly in onclick handlers. Malicious shop names with quotes can break out and inject code.

Attack Example: Shop name like abc', undefined, 1); alert('xss'); // could inject arbitrary code

Recommended Fix: Use data attributes combined with addEventListener instead of inline onclick handlers

BUG #8: Missing Error Handling in Renewal Processing

Severity: MEDIUM | **File:** script.js, Lines 1220-1230

Problem: Multiple async database operations lack comprehensive error handling. If deleteGoogleDriveFolder fails, shop may be deleted from database without proper cleanup.

Recommended Fix: Wrap critical operations in try-catch blocks to ensure consistency

BUG #9: Hardcoded Firebase Credentials in Frontend

Severity: MEDIUM | **File:** script.js, Lines 13-21

Problem: Firebase API keys and project IDs are exposed in frontend source code. These are visible to any user viewing the page source.

Impact: Attackers can potentially access your Firebase instance

Recommended Fix: Implement strict Firebase Security Rules to restrict unauthorized access, even with exposed credentials

General Recommendations

1. Enable Content Security Policy (CSP) HTTP header to mitigate XSS attacks

2. Input Validation: Sanitize and validate all user inputs before processing

3. Error Handling: Add try-catch blocks around all critical async operations

4. Code Review: Implement peer review process for security-critical code

5. HTTPS Only: Ensure all communications use HTTPS encryption

6. Firebase Security Rules: Implement strict rules to control data access

7. Regular Security Audits: Perform regular security assessments of the codebase

8. Use Security Testing Tools: Implement automated security scanning in CI/CD pipeline

This report was generated by GitHub Copilot Code Analysis on 23/08/2026 19:48:02